



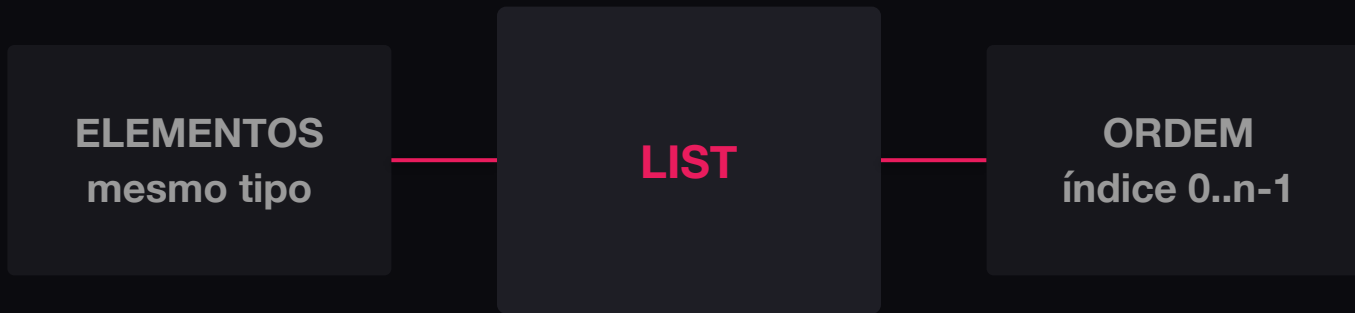
DART · ARRAY

Dominando o Array

Como criar, acessar e transformar coleções em Dart — List, Map e Set.

O QUE É UMA LIST?

A forma como Dart representa um array: uma coleção ordenada e tipada de elementos.



- Permite elementos repetidos e cresce dinamicamente.
- Cada elemento tem uma posição fixa, acessada por índice.
- É a base para armazenar múltiplos valores relacionados.

A ANATOMIA BÁSICA

```
List<String> frutas = [  
    'maçã',  
    'pera',  
    'banana',  
];
```

List<T> – O Tipo

Declara uma lista tipada; T define o tipo dos elementos.

[] – O Literal

Cria a lista diretamente, com os valores entre colchetes.

CRIANDO VAZIA E POPULANDO: ADD()

Uma lista também pode começar vazia e crescer aos poucos.

```
List<String> profissao = [];  
  
profissao.add('programador');  
profissao.add('médico');
```

add() insere um novo elemento sempre no final da lista — o tamanho cresce a cada chamada.

ACESSANDO E ALTERANDO POR ÍNDICE

O índice é a posição do elemento dentro da lista.

```
final comidas = ['arroz', 'feijão'];  
print(comidas[0]); // arroz  
comidas[1] = 'macarrão';  
print(comidas[1]); // macarrão
```

O índice sempre começa em 0. Acessar uma posição fora do intervalo lança um `RangeError`.

TAMANHO FIXO: LIST.FILLED

Cria uma lista com N posições já preenchidas por um valor inicial.

```
List<String> comidas = List.filled(3, "");  
comidas[0] = "arroz";  
comidas[1] = "feijão";  
comidas[2] = "macarrão";
```

As posições podem ser alteradas depois, mas o tamanho da lista não muda — diferente de uma lista criada com `[]`.

ADICIONAR, INSERIR E REMOVER

Três operações essenciais para manipular os elementos de uma lista.

add(valor)

Adiciona um novo elemento sempre ao final da lista.

insert(i, valor)

Insere um elemento numa posição específica, deslocando os demais para frente.

remove(valor)

Remove a primeira ocorrência do valor — não do índice.

OUTRAS COLEÇÕES: MAP E SET

Nem toda coleção precisa de índice numérico ou permite duplicados.

Map<K, V> — Chave-valor

Cada valor é acessado por uma chave: mapa['chave'].

Set<T> — Sem repetição

Ignora automaticamente valores duplicados ao inserir.

```
final usuario = {'nome': 'Ana', 'idade': 23};  
final tags = {'dart', 'flutter', 'dart'};  
// tags: {dart, flutter} – o segundo 'dart' foi ignorado
```

FILTRANDO E TRANSFORMANDO: WHERE E MAP

Dois métodos que percorrem a coleção sem que você escreva o laço.

```
final numeros = [1, 2, 3, 4, 5, 6];  
final pares = numeros.where((n) => n % 2 == 0);  
final dobro = numeros.map((n) => n * 2);
```

where filtra elementos que satisfazem uma condição. map transforma cada elemento, retornando um novo Iterable.

EXEMPLO COMENTADO

```
void main() {  
    List<String> frutas = ['maçã', 'pera', 'banana'];  
    List<int> numeros = [1, 2, 3, 4, 5];  
  
    List<String> profissao = [];  
    profissao.add('programador');  
    profissao.add('médico');  
  
    List<String> comidas = List.filled(3, "");  
    comidas[0] = "arroz";  
    comidas[1] = "feijão";  
    comidas[2] = "macarrão";  
  
    print("Lista de frutas: $frutas");  
    print("Lista de números: $numeros");  
    print("Lista de profissões: $profissao");  
    print("Lista fixa de comidas: $comidas");  
}
```

SAÍDA ESPERADA

```
$ Lista de frutas: [maçã, pera, banana]  
$ Lista de números: [1, 2, 3, 4, 5]  
$ Lista de profissões: [programador, médico]  
$ Lista fixa de comidas: [arroz, feijão, macarrão]
```

Cada print interpola a lista inteira com \$variavel — o toString() de uma List já formata os colchetes.

LIST, MAP OU SET?

	List<T>	Map<K, V>	Set<T>
Acesso	Por índice: lista[i]	Por chave: mapa['k']	Não há índice
Duplicados	Permite repetição	Chaves são únicas	Ignora valores repetidos
Uso ideal	Ordem e posição importam	Associar um dado a um nome	Garantir valores únicos

MUNDO REAL: RENDERIZANDO UMA LISTA DE WIDGETS

O índice da lista decide qual item construir na tela.

```
ListView.builder(  
  itemCount: produtos.length,  
  itemBuilder: (context, i) => ItemCard(produtos[i]),  
)
```

ListView.builder usa o índice para construir sob demanda apenas os itens visíveis na tela — essencial para listas longas.

A ARTE DO CÓDIGO LIMPO

✓ Certo

```
if (lista.isEmpty) return;  
final primeiro = lista.first;
```

- Verifica isEmpty antes de acessar.
- Usa first em vez de lista[0].
- Evita RangeError em lista vazia.

✗ Errado

```
final primeiro = lista[0];  
// quebra se a lista estiver vazia
```

- Acessa o índice sem checar o tamanho.
- RangeError em tempo de execução.
- Só falha quando roda com dados reais.

JUNTANDO TUDO

```
List<int> paresAcimaDe(List<int> numeros, int minimo) {  
    return numeros  
        .where((n) => n % 2 == 0)  
        .where((n) => n > minimo)  
        .toList();  
}
```

where (encadeado)

Cada where filtra sobre o resultado do where anterior.

.toList()

Converte o Iterable resultante de volta em List.

MINI-RESUMO (CHEAT SHEET)

List<T>

Coleção ordenada e tipada; representa um array.

lista[i]

Acessa ou altera um elemento pelo índice, a partir de 0.

List.filled(n, v)

Cria uma lista de tamanho fixo já preenchida.

Map / Set

Chave-valor / valores únicos, sem duplicar.

where / map

Filtra elementos / transforma cada elemento.



DIVIRTA-SE!

Profº Vinny Albuquerque
profvinny.albuquerque@fiap.com.br
@mvalbuquerque